

โครงสร้างกิจกรรม: จาก User Journey สู่โค้ดจริงด้วย AI (โปรเจกต์ต่อเนื่องหลายสัปดาห์)

เอกสารนี้ออกแบบมาเพื่อใช้เป็นแนวทางให้อาจารย์วางโครงสร้างกิจกรรมที่ให้นิสิต นำ **User Journey** ที่ตนเองออกแบบไว้ มา **prompt AI** ให้ช่วยเขียนโค้ด โดยเริ่มจากกิจกรรม kickoff ในช่วงบ่ายวันนี้ และขยายต่อเนื่องเป็นโปรเจกต์ full-stack ที่ deploy ขึ้นใช้งานจริงบนอินเทอร์เน็ตได้ภายในหลายสัปดาห์

บริบทที่ใช้ออกแบบเอกสารนี้: นิสิตพื้นฐานเริ่มต้น (ยังไม่เคยเขียนโค้ดจริงจิง), กิจกรรมบ่ายนี้เป็นจุดเริ่มโปรเจกต์ที่จะทำต่อเนื่องหลายสัปดาห์, และเป้าหมายปลายทางคือต้อง deploy ขึ้นอินเทอร์เน็ตได้จริง (มี URL ใช้งานได้)

อัปเดต Stack ตามที่อาจารย์กำหนด: Backend ใช้ **FastAPI (Python)** ซึ่งเป็นสิ่งที่อาจารย์จะสอนอยู่แล้ว, Frontend ใช้ **HTML/CSS/JavaScript ธรรมดา** (ไม่ใช่ framework เช่น React/Vue) เพื่อลดความซับซ้อน และควบคุมการรันทั้งระบบ (frontend+backend+database) ด้วย **Docker/Docker Compose** เป็นมาตรฐานเดียวกัน ทุกกลุ่ม เอกสารส่วนที่เหลือทั้งหมดปรับให้สอดคล้องกับ stack นี้แล้ว

1. ภาพรวมและเป้าหมาย

เป้าหมายบ่ายนี้ (Kickoff Session): ทุกกลุ่มมี (ก) repository บน GitHub (ข) โครงสร้างโปรเจกต์ (scaffold) ที่รันได้ (ค) หน้าจอเริ่มต้นที่ AI ช่วยสร้างจาก User Journey ของกลุ่ม พร้อม mock data (ง) เว็บไซต์ deploy ขึ้นจริงแล้ว (แม้ยังเป็นแค่ frontend เปล่าๆ) เพื่อให้นิสิตเห็น "ของจริงบนอินเทอร์เน็ต" ตั้งแต่วันแรก เป็นแรงจูงใจในการทำต่อ

เป้าหมายปลายทาง (สัปดาห์ที่ 5-6): ระบบ full-stack ที่มี frontend + backend API + ฐานข้อมูลจริง + ระบบล็อกอิน (ถ้าเข้ากับ journey) deploy ใช้งานได้จริงบนอินเทอร์เน็ต ตรงตาม User Journey ที่ออกแบบไว้ในสัปดาห์แรก

หลักการสำคัญ 3 ข้อที่ควรย้ำกับนิสิตตั้งแต่ต้น:

1. **เริ่มจาก URL จริงตั้งแต่วันแรก** — deploy scaffold เปล่าๆ ขึ้นเน็ตก่อน แล้วค่อยๆ เติมนิเจอร์ทีละสัปดาห์ (ดีกว่าทำทุกอย่างในเครื่องแล้วมา deploy ที่เดียวตอนท้าย ซึ่งมักพังและแก้ไม่ทัน)
 2. **แตกงานเป็นขั้นเล็ก แล้ว prompt AI ทีละขั้น** — ไม่ prompt ให้ AI สร้างทั้งระบบในครั้งเดียว เพราะนิสิตมือใหม่จะตรวจสอบ/แก้โค้ดจำนวนมากไม่ทัน
 3. **ทุกกลุ่มใช้โครงสร้าง (scaffold) เดียวกัน** — อาจารย์เตรียม starter template ให้ทุกกลุ่มเริ่มจากจุดเดียวกัน ลดปัญหาต่างกลุ่มต่างสไตล์จนช่วยกันแก้ปัญหาไม่ได้
-

2. Tech Stack ที่แนะนำ (และเหตุผลที่เหมาะสมกับบริบทนี้)

ชั้นระบบ (Layer)	เครื่องมือที่แนะนำ	เหตุผล
Frontend	HTML + CSS + JavaScript ธรรมดา (ไม่มี framework)	ไม่ต้องเรียนรู้ syntax/build tool เพิ่ม, เปิดไฟล์ก็รันได้ทันที, ใช้ <code>fetch()</code> เรียก API ตรงๆ พอ สำหรับ CRUD ทั่วไป และ AI generate ได้ตรงไปตรงมาที่สุด
Backend	FastAPI (Python)	ตรงกับสิ่งที่อาจารย์จะสอน, เขียน ภาษาเดียวกับที่นิสิตคุ้นเคย, มี auto-docs (Swagger UI ที่ <code>/docs</code>) ให้ทดสอบ API ได้ทันที โดยไม่ต้องพึ่งเครื่องมือภายนอก
ฐานข้อมูล	PostgreSQL (รันเป็น container ผ่าน Docker Compose)	ข้อมูลจริง เป็นมาตรฐานเดียวกับ ที่ใช้ deploy จริงได้ ไม่ต้องติด ตั้งอะไรเพิ่มในเครื่อง เพราะ Docker จัดการให้
ORM (ตัวช่วยคุยกับ DB)	SQLModel (หรือ SQLAlchemy)	เขียนโดยผู้พัฒนา FastAPI คน เดียวกัน ใช้ syntax คล้าย Pydantic ที่ FastAPI ใช้อยู่แล้ว เรียนรู้เพิ่มน้อย, AI generate model/query ได้แม่นยำ
การจัดการรันระบบ	Docker + Docker Compose	สั่งคำสั่งเดียว (<code>docker compose up</code>) ได้ทั้ง frontend + backend + database ครบ ทุกเครื่องของทุกคน/ทุกกลุ่มได้ ผลลัพธ์เหมือนกันเป๊ะ ไม่มีปัญหา "ในเครื่องผมรัน แต่เพื่อนรันไม่ ได้"
Version Control	Git + GitHub	จำเป็นสำหรับทำงานกลุ่มและ เชื่อมต่อระบบ deploy อัตโนมัติ (CI/CD)
Deploy	แพลตฟอร์มที่รองรับ Docker โดยตรง เช่น Render หรือ Railway	อ่าน Dockerfile/Docker Compose เดียวกับที่ใช้ตอน พัฒนาไปรันบน production ได้

ชั้นระบบ (Layer)	เครื่องมือที่แนะนำ	เหตุผล
		เลย ไม่ต้องเขียนขั้นตอน deploy แยกต่างหาก

ทำไม stack นี้ถึง "ไม่ซับซ้อน" แต่ยังสมจริง:

- Frontend ไม่มี build step, ไม่มี state management ให้งง — เน้นให้นิสิตเข้าใจ "หน้าเว็บเรียก API แล้วเอาข้อมูลมาแสดง" ซึ่งเป็นแกนของ full-stack development
- Backend เขียนภาษาเดียวกับที่เรียนในห้อง (Python) ไม่ต้องสลับ context ไปมาระหว่าง Python กับ JavaScript ฝั่ง backend
- Docker ทำให้ "โครงสร้างระบบ" เหมือนกันตั้งแต่เครื่องนิสิตจนถึง production จริง ลดปัญหาเรื่อง environment ที่มักเป็นจุดที่มือใหม่ติดขัดมากที่สุดตอน deploy

3. โครงสร้าง Repository และ Docker Compose ที่แนะนำ (Scaffold ดั้งเดิม)

ให้อาจารย์เตรียม **GitHub template repository** ล่วงหน้า 1 ชุด แล้วให้ทุกกลุ่มกด "Use this template" เพื่อเริ่มจากโครงเดียวกัน โดย FastAPI จะทำหน้าที่ทั้ง serve หน้าเว็บ (static files) และเปิด API endpoint ในตัวเดียวกัน ไม่ต้องแยก server สองตัว:

project-name/

├── backend/

| ├── app/

| | ├── main.py # จุดเริ่มแอป, mount static files (frontend) + include routers

| | ├── routers/ # แยก endpoint เป็นหมวดตาม User Journey เช่น bookings.py, users.py

| | ├── models.py # SQLAlchemy: โครงสร้างตาราง/ฐานข้อมูล

| | ├── schemas.py # Pydantic: รูปแบบข้อมูลรับ-ส่งผ่าน API

| | ├── database.py # การเชื่อมต่อฐานข้อมูล PostgreSQL

| | └── crud.py # ฟังก์ชันอ่าน/เขียนข้อมูลจริง

| └── requirements.txt # รายชื่อ Python package ที่ต้องติดตั้ง

```
|   └─ Dockerfile      # สร้าง image ของ backend
|
| └─ frontend/        # HTML/CSS/JS ธรรมดา ไม่มี build step
|
|   └─ index.html
|
|   └─ pages/          # หน้าจอ .html ตามขั้นตอนใน User Journey
|
|     └─ select-datetime.html
|
|     └─ css/
|
|       └─ style.css
|
|       └─ js/
|
|         └─ api.js      # ฟังก์ชัน fetch() เรียก FastAPI (mock data ได้ในสไลด์แรก)
|
| └─ docs/
|
|   └─ user-journey.md  # ไฟล์ User Journey ของกลุ่ม (นำมาจากสไลด์ก่อน)
|
|   └─ er-diagram.png   # แผนภาพฐานข้อมูล (ทำสไลด์ที่ 2)
|
|   └─ api-spec.md      # รายการ API endpoint ทั้งหมด
|
| └─ docker-compose.yml  # สั่งรัน backend + frontend + database พร้อมกันด้วยคำสั่งเดียว
|
| └─ .env.example        # ตัวอย่างตัวแปรลับ (ห้าม commit .env จริง)
|
| └─ .gitignore
|
| └─ README.md           # วิธีติดตั้ง/รันโปรเจกต์ + ลิงก์เว็บที่ deploy แล้ว
```

ตัวอย่างโครงสร้าง `docker-compose.yml`

services:

web:

build: ./backend

ports:

- "8000:8000"

volumes:

- ./backend/app:/app/app # แก่โค้ดฝั่งเครื่อง เห็นผลทันทีไม่ต้อง build ใหม่
- ./frontend:/app/frontend # FastAPI serve ไฟล์ frontend จากโฟลเดอร์นี้

env_file: .env

depends_on:

- db

db:

image: postgres:16

environment:

POSTGRES_USER: appuser

POSTGRES_PASSWORD: apppassword

POSTGRES_DB: appdb

volumes:

- db_data:/var/lib/postgresql/data

ports:

- "5432:5432"

volumes:

db_data:

นิสิตทุกกลุ่มใช้คำสั่งเดียวกันตลอดทั้งเทอม: **docker compose up** เพื่อรันทั้งระบบในเครื่องตัวเอง และ **docker compose down** เพื่อปิด — ไม่ต้องจำวิธีติดตั้ง Python/Postgres แยกเอง

สิ่งที่ต้องเน้นย้ำเรื่องโครงสร้างนี้กับนิสิต:

- โฟลเดอร์ `frontend/pages/` ควรตั้งชื่อไฟล์ตาม "ขั้นตอน" ในตาราง User Journey โดยตรง (เช่น journey มีขั้นตอน "หน้าเลือกวัน-เวลา" ก็ตั้งชื่อไฟล์ `select-datetime.html`) เพื่อให้ prompt AI และตรวจงานง่าย ตรงกับเอกสารที่ทำไว้ก่อนหน้า
- โฟลเดอร์ `backend/app/routers/` ควรตั้งชื่อไฟล์ตาม Entity หลักของระบบ (เช่นระบบจองคิว → `bookings.py`) ให้สอดคล้องกับหัวข้อที่ 8
- `.env` (คีย์ลับ/รหัสฐานข้อมูล) ต้องไม่ถูก push ขึ้น GitHub เด็ดขาด — ต้องสอนเรื่องนี้ตั้งแต่วันแรก เพราะเป็นความผิดพลาดที่พบบ่อยที่สุดของมือใหม่ (ใส่ `.env` ไว้ใน `.gitignore` ตั้งแต่ตอนสร้าง repo)

4. Git/GitHub Workflow สำหรับทำงานกลุ่ม

หัวข้อ	แนวทางที่แนะนำ
จำนวน repo	1 repository ต่อ 1 กลุ่ม (ไม่ใช่ต่อคน) เพื่อฝึกทำงานร่วมกันจริง
Branch หลัก	<code>main</code> = โค้ดที่ deploy จริงเท่านั้น ห้าม push ตรงเข้า <code>main</code>
Branch ทำงาน	แต่ละคน/แต่ละฟีเจอร์แตก branch เช่น <code>feature/login-page</code> แล้วส่ง Pull Request กลับเข้า <code>main</code>
การตรวจงาน	ใช้ Pull Request เป็นจุดที่อาจารย์/เพื่อนในกลุ่มรีวิวโค้ดก่อน merge
Commit message	กำหนดรูปแบบง่ายๆ เช่น <code>feat</code> : เพิ่มหน้าจองคิว, <code>fix</code> : แก้บั๊กปุ่มยืนยันไม่ทำงาน
ความถี่	อย่าให้ commit บ่อยๆ (อย่างน้อยทุกครั้งที่ทำฟีเจอร์ย่อยเสร็จ) ไม่ใช่รวบยอดตอนจบสัปดาห์
ไฟล์ที่ต้อง ignore	ใส่ <code>.env</code> , โฟลเดอร์ <code>__pycache__/, *.pyc</code> และข้อมูล Docker volume ไว้ใน <code>.gitignore</code> ตั้งแต่ commit แรก

5. Roadmap โปรเจกต์ (ปรับจำนวนสัปดาห์ตามปฏิทินจริงได้)

ช่วงเวลา	เป้าหมายหลัก	สิ่งที่ต้องส่ง (Deliverable)	Checkpoint ที่อาจารย์ตรวจ
สัปดาห์ 1 (บ่ายนี้)	Kickoff: ตั้ง repo+scaffold, docker compose up ให้รันได้ในเครื่อง, prompt AI สร้างหน้า HTML 2-3 หน้า + FastAPI endpoint คืนค่า mock data, deploy ครั้งแรก	URL เรียบ (frontend+FastAPI คืน mock data)	docker compose up รันได้ไม่มี error, deploy สำเร็จ, โครงสร้างไฟล์ตรงตาม scaffold
สัปดาห์ 2	ออกแบบฐานข้อมูล (data model) จาก User Journey ด้วย SQLAlchemy + สร้าง backend API จริงเชื่อม PostgreSQL	ER diagram, models.py , API endpoint อย่างน้อย 2 เส้นทาง (GET/POST) ทดสอบผ่าน Swagger UI (/docs)	เรียก API ผ่านหน้า /docs ของ FastAPI แล้วบันทึก/อ่านข้อมูลจริงจาก Postgres ได้
สัปดาห์ 3	เชื่อม frontend (fetch จาก js/api.js) เข้ากับ backend จริง (เลิกใช้ mock data)	ระบบ CRUD พื้นฐาน ทำงานครบวงจร	ข้อมูลที่กรอกใน frontend บันทึกลงฐานข้อมูลจริงได้
สัปดาห์ 4	เพิ่มระบบล็อกอิน/สิทธิ์ผู้ใช้ (ถ้า journey ต้องการ) + จัดการ Edge Cases จากเอกสารเดิม	ระบบ auth ทำงาน, มีการจัดการ error เบื้องต้น	ทดสอบ edge case อย่างน้อย 3 กรณีจากเอกสาร User Journey
สัปดาห์ 5	Deploy เวอร์ชันเต็ม (frontend+backend+DB) + ทดสอบผู้ใช้จริง (usability test เพื่อนกลุ่มอื่น)	เว็บใช้งานได้จริงครบทุกฟีเจอร์หลัก	เพื่อนกลุ่มอื่นทดลองใช้งานตาม User Journey ได้จบ flow โดยไม่ติดขัด
สัปดาห์ 6	ปรับปรุงจาก feedback + นำเสนอโปรเจกต์	Slide + Demo สด + README สมบูรณ์	นำเสนอ 5-10 นาที พร้อมตอบคำถาม

6. กิจกรรมช่วงบ่ายนี้ (Kickoff Session) — ขั้นตอนที่ให้นิสิตทำตามทีละขั้น

1. แบ่งกลุ่ม (3-4 คน) และยืนยันหัวข้อ — แต่ละกลุ่มเลือก 1 อุตสาหกรรม/ระบบจาก User Journey ที่ทำไว้ (จากเอกสาร 24 ระบบ หรือ journey ที่ตัวเองออกแบบเพิ่มเติม)
 2. กรอก Prompt Template (ดูหัวข้อที่ 7) โดยคัดลอกเนื้อหา Persona / ขั้นตอน / Edge Case / Acceptance Criteria จากเอกสาร User Journey ของกลุ่มตัวเองมาใส่
 3. สร้าง Repository จาก GitHub template ที่อาจารย์เตรียมไว้ (กด "Use this template") ตั้งชื่อ repo ตามชื่อระบบของกลุ่ม แล้ว clone ลงเครื่อง
 4. ตรวจสอบว่า Docker ใช้งานได้ — รันคำสั่ง `docker compose up` จาก scaffold ตั้งต้น ต้องเห็นหน้าเว็บที่ <http://localhost:8000> และหน้า Swagger API ที่ <http://localhost:8000/docs> ก่อนเริ่ม prompt ใดๆ (แก้ปัญหาเรื่องเครื่อง/การติดตั้งให้เสร็จก่อน จะได้ไม่เสียเวลาตอนกำลังเขียนโค้ด)
 5. Prompt AI รอบที่ 1 — ขอให้ AI สร้างเฉพาะ "หน้าจอหลัก 2-3 หน้าแรก" ของ journey ก่อน (ไฟล์ `.html` ใน `frontend/pages/`) พร้อม 1 FastAPI endpoint ที่คืนค่า mock data ที่ตรงกับบริบท (ดูตัวอย่างในหัวข้อที่ 7) — ไม่ต้องทำทั้งระบบทีเดียว
 6. ตรวจสอบผลใน browser ตรวจสอบว่าหน้าจอที่ได้ตรงกับ journey ที่ออกแบบไว้หรือไม่ (ไฟล์ที่แก้ในเครื่องจะเห็นผลทันทีเพราะ docker-compose mount volume ไว้แล้ว ไม่ต้อง build ใหม่) ถ้าไม่ตรง ให้ prompt AI แก่ทีละจุด (ไม่ใช่ prompt ใหม่ทั้งหมด)
 7. Push ขึ้น GitHub แล้วเชื่อมต่อบริการที่รองรับ Docker (เช่น Render/Railway — ดูหัวข้อที่ 9) เพื่อให้ได้ URL ใช้งานจริงทันที
 8. บันทึก URL และ prompt ที่ใช้ ลงใน `README.md` ของ repo เพื่อใช้ต่อในสัปดาห์หน้า และเพื่อให้อาจารย์ตรวจสอบกระบวนการ prompt (ไม่ใช่แค่ผลลัพธ์)
 9. สรุปปิดท้าย 5 นาที/กลุ่ม — โขว์ URL จริงที่ deploy ได้ และปัญหาที่เจอระหว่าง prompt AI (เพื่อแลกเปลี่ยนบทเรียนกับกลุ่มอื่น)
-

7. Prompt Template สำหรับนิสิต (Structured Prompting Framework)

แจก template นี้ให้นิสิตทุกกลุ่ม "กรอกให้ครบ" ก่อน copy ไปวางใน AI เพราะการให้บริบทครบถ้วนคือปัจจัยสำคัญที่สุดที่ทำให้ AI เขียนโค้ดได้ตรงและแก้ไขง่าย

แม่แบบ Prompt (Fill-in-the-blank)

บทบาทของคุณ: คุณเป็นนักพัฒนา Full-stack ที่กำลังช่วยนิสิตมือใหม่สร้างเว็บแอปจาก User Journey

เทคโนโลยีที่ต้องใช้:

- Frontend: HTML + CSS + JavaScript ธรรมดา (ไม่ใช่ framework ใดๆ), เรียก API ด้วย fetch()
- Backend: FastAPI (Python)
- ระบบทั้งหมดรันผ่าน Docker Compose อยู่แล้ว ไม่ต้องแนะนำวิธีติดตั้ง Python/Node เอง
- โครงสร้างไฟล์: [วางโครงสร้าง scaffold จากหัวข้อที่ 3]

บริบทของระบบ:

- อุตสาหกรรม/ระบบ: [ชื่อระบบ เช่น "จองคิวร้านเสริมสวย/สปา"]
- Persona: [วางจากเอกสาร User Journey]
- เป้าหมายของผู้ใช้: [วางจากเอกสาร]

ขอบเขตงานที่ต้องการในรอบนี้ (ทำทีละส่วน อย่าขอทั้งหมดทีเดียว):

- สร้างเฉพาะหน้าจอ HTML: [ระบุ 2-3 ขั้นตอนจากตาราง User Journey เช่น "หน้าค้นหาร้าน" และ "หน้าเลือกวัน-เวลา"]
- สร้าง FastAPI endpoint ที่คืนค่าเป็น mock data (list ของ dict ใน Python) ไปก่อน ยังไม่ต้องเชื่อมฐานข้อมูลจริง
- ให้นำหน้า HTML เรียก endpoint นี้ด้วย fetch() แล้วแสดงผลด้วย JavaScript ธรรมดา
- ต้องรองรับ Edge Case ต่อไปนี้ในหน้า UI (แสดงข้อความแจ้งเตือนที่เหมาะสม): [วาง Edge Cases ที่เกี่ยวข้องจากเอกสาร]

เกณฑ์ตรวจรับงาน (Acceptance Criteria) ที่โค้ดต้องผ่าน:

[วาง Acceptance Criteria ที่เกี่ยวข้องจากเอกสาร]

ข้อกำหนดการเขียนโค้ด:

- แบ่งไฟล์ตามโครงสร้างที่กำหนด (ไฟล์ .html แยกต่อหน้า, JavaScript แยกไฟล์ใน js/, FastAPI route แยกไฟล์ใน routers/) ห้ามเขียนรวมในไฟล์เดียว
- ใส่คอมเมนต์อธิบายส่วนสำคัญเป็นภาษาไทยสั้นๆ เพื่อให้คนอื่นอ่านเข้าใจ
- อธิบายทีละขั้นตอนก่อนว่าจะสร้าง/แก้ไฟล์อะไรบ้าง แล้วค่อยให้โค้ดแต่ละไฟล์

- หลังจบให้สรุปว่าต้องแก้ไฟล์ไหนเพิ่ม (เช่น เพิ่ม route ใน main.py) และทดสอบอย่างไรผ่าน Swagger UI (`/docs`)

ตัวอย่าง Prompt ที่กรอกสมบูรณ์แล้ว (อ้างอิงระบบที่ 20: จองคิวร้านเสริมสวย/สปา)

บทบาทของคุณ: คุณเป็นนักพัฒนา Full-stackที่กำลังช่วยนิสิตมือใหม่สร้างเว็บแอปจาก User Journey

เทคโนโลยีที่ต้องใช้: HTML/CSS/JavaScript ธรรมดา (frontend) + FastAPI (backend) รันผ่าน Docker Compose อยู่แล้ว

โครงสร้างไฟล์: frontend/pages/, frontend/js/, backend/app/routers/

บริบทของระบบ: ระบบจองคิวร้านเสริมสวย/สปา

Persona: "คุณมินท์" อายุ 31 ปี นัดทำผม/สปาเป็นประจำทุกเดือน ต้องการจองช่างที่ถนัดคนเดิม

เป้าหมาย: จองคิวช่าง/บริการที่ต้องการ ในวันเวลาที่สะดวก โดยไม่ต้องโทรศัพท์

ขอบเขตงานรอบนี้:

- สร้างหน้า frontend/pages/search-salon.html แสดงรายการร้านแบบการ์ด พร้อมช่องค้นหา/กรองตามพื้นที่
- สร้างหน้า frontend/pages/select-datetime.html แสดงปฏิทินอย่างง่ายและช่องเวลาวางของช่างแต่ละคน
- สร้าง backend/app/routers/salons.py มี endpoint GET /api/salons และ GET /api/salons/{id}/slots คืนค่า mock data เป็น list ของ dict
- ให้ js/api.js เรียกทั้งสอง endpoint ด้วย fetch() แล้วแสดงผลในหน้า HTML

Edge Case ที่ต้องรองรับใน UI:

- ถ้าช่างที่เลือกไม่มีคิวว่างในวันนั้น ให้แสดงข้อความแนะนำช่างคนอื่น/วันอื่น
- ถ้าผู้ใช้เลือกเวลาที่ผ่านมาแล้ว (อดีต) ให้ปิดกั้นไม่ให้เลือกได้

Acceptance Criteria:

- ลูกค้าเห็นคิวว่างของช่างแบบชัดเจน แยกตามช่างแต่ละคน
- มีการแสดงผลที่ใช้งานได้ดีบนมือถือ (responsive CSS พื้นฐาน)

ข้อกำหนดการเขียนโค้ด: แบ่งไฟล์ตามโครงสร้างที่กำหนด ใส่คอมเมนต์ภาษาไทย อธิบายก่อนแล้วค่อยให้โค้ด และสรุปวิธีทดสอบผ่าน /docs ท้ายสุด

เทคนิคการ Prompt ต่อเนื่อง (สำหรับตอนแก้บั๊ก/ต่อยอด)

- เวลาโค้ดมีปัญหา ให้ **แนบ error message** เต็มๆ และบอกว่า "กำลังทำอะไรอยู่ตอนที่เจอ error" แทนการพิมพ์แค่ "มันไม่ทำงาน"
- ขอให้ AI **อธิบายโค้ดที่เพิ่งสร้างให้ฟัง** ก่อน commit ทุกครั้ง เพื่อให้แน่ใจสิ่งที่ตัวเองจะส่งไม่ใช่แค่ copy-paste
- เมื่อจะเพิ่มฟีเจอร์ใหม่ ให้บอก AI ด้วยว่า **ไฟล์ไหนมีอยู่แล้วบ้าง** (วางโครงสร้างปัจจุบันให้ดู) เพื่อให้ AI สร้างไฟล์ซ้ำหรือเขียนทับของเดิมโดยไม่ตั้งใจ

8. แนวทาง Backend + Database (สำหรับสัปดาห์ที่ 2 เป็นต้นไป)

ขั้นตอนแปลง User Journey เป็นฐานข้อมูล:

1. ให้นิสิตไล่อ่านตาราง "ขั้นตอน" ใน User Journey ของตัวเอง แล้วขีดเส้นใต้ "คำนาม" ทั้งหมด (เช่น ร้าน, ช่าง, คิว, ลูกค้า, การจอง) — คำนามเหล่านี้คือ Entity/ตารางในฐานข้อมูลตัวตั้งต้น
2. ให้ prompt AI ช่วยออกแบบ schema จาก Entity ที่ได้ ตัวอย่าง prompt:

จาก User Journey นี้ [วาง journey เต็ม] ช่วยออกแบบ Database Schema โดยใช้ SQLAlchemy (Python)

โดยระบุ Entity หลัก ความสัมพันธ์ระหว่างตาราง (one-to-many/many-to-many)

และอธิบายเหตุผลการออกแบบแต่ละจุดสั้นๆ เป็นภาษาไทย ก่อนให้โค้ด models.py

3. ตรวจสอบ schema ที่ AI ออกแบบมาด้วยตัวเอง (อย่าเชื่อทันที) โดยเทียบกับ Edge Cases ในเอกสาร User Journey ว่าโครงสร้างรองรับกรณีเหล่านั้นหรือไม่ (เช่น ระบบจองคิวต้องการการจองซ้อนเวลาเดียวกันได้)
4. ให้ AI generate REST API endpoint ตาม "ขั้นตอน" ใน journey ที่ละหมวด โดยแยกไฟล์ใน `backend/app/routers/` เช่น หมวดการจอง → `bookings.py` มี `GET /api/bookings`, `POST /api/bookings`, `PUT /api/bookings/{id}/cancel` แล้วนำ router ไป `include_router()` ใน `main.py`
5. เมื่อแก้ `models.py` ให้ prompt AI ช่วยเขียนคำสั่งสร้าง/อัปเดตตารางในฐานข้อมูล (เช่น ใช้ Alembic สำหรับ migration หรือ `SQLModel.metadata.create_all()` แบบง่ายในช่วงต้นเทอม) และทดสอบผ่าน `docker compose up` ทุกครั้งที่แก้ schema

ข้อดีของ FastAPI ในขั้นตอนนี้: ทุก endpoint ที่สร้างจะปรากฏในหน้า Swagger UI (<http://localhost:8000/docs>) โดยอัตโนมัติ นิสิตทดสอบยิง API ได้จากหน้าเว็บนี้เลย โดยไม่ต้องใช้เครื่องมือภายนอกอย่าง Postman ก็ได้ในช่วงต้น

ข้อแนะนำเรื่อง API:

- ตั้งชื่อ endpoint ให้สื่อความหมายตรงกับค่าในตาราง User Journey เพื่อให้อาจารย์ตรวจสอบย้อนกลับไปยัง journey เดิมได้ง่าย
- ทุก endpoint ที่แก้ไข/ลบข้อมูล ต้อง prompt ให้ AI ใส่การตรวจสอบสิทธิ์ (authorization) และ validate ข้อมูลนำเข้าเสมอ ไม่ใช่แค่ happy path

9. แนวทาง Deployment

เพราะทั้งระบบถูกควบคุมด้วย Docker อยู่แล้ว ข้อดีคือ **สภาพแวดล้อมตอน deploy จริงเหมือนกับตอนพัฒนาในเครื่องทุกประการ** — ไม่ต้องเขียนขั้นตอน deploy แยกต่างหาก แค่ให้บริการ hosting รัน Dockerfile/Docker Compose ตัวเดียวกัน

Stack	บริการ Deploy	ขั้นตอนโดยสรุป
Backend + Frontend (FastAPI serve ทั้งคู่ในตัวเดียว)	Render (ประเภท "Docker" service) หรือ Railway	เชื่อมบัญชี GitHub → เลือก repo → บริการจะอ่าน Dockerfile ในโฟลเดอร์ backend/ แล้ว build/deploy ให้อัตโนมัติทุกครั้งที่เรา push เข้า main
Database	Managed PostgreSQL ของบริการเดียวกัน (Render PostgreSQL / Railway PostgreSQL) หรือ Supabase/Neon (free tier)	สร้างฐานข้อมูลจากหน้าเว็บบริการ → คัดลอก connection string มาตั้งเป็น environment variable DATABASE_URL บนหน้าตั้งค่าของบริการ (ไม่ใช่ในโค้ด/ไม่ใช่ค่าเดียวกับ container Postgres ตอน dev)
Environment Variables	ตั้งค่าในหน้าเว็บของบริการ deploy โดยตรง	ค่าที่ใช้ตอน dev อยู่ใน .env (ไม่ commit), ค่าที่ใช้ตอน production ตั้งแยกในระบบของ Render/Railway เพื่อความปลอดภัย

ทำไมจึงไม่ต้องแยก deploy frontend/backend เป็นสองที่: เพราะ FastAPI mount โฟลเดอร์ **frontend/** เป็น static files ในตัว (ตามตัวอย่าง **docker-compose.yml** ในหัวข้อที่ 3) ทำให้ deploy container เดียวได้ทั้งเว็บหน้าบ้านและ API ลดความซับซ้อนของขั้นตอน deploy ลงไปอีกขั้นหนึ่ง เหมาะกับนิสิตเริ่มต้นมาก

ข้อควรย้ำ: ให้นิสิต deploy ตั้งแต่สัปดาห์ 1 (แม้ endpoint จะคืนแค่ mock data) แล้ว deploy ซ้ำทุกสัปดาห์เมื่อมีของใหม่ ดีกว่าปล่อยไว้ deploy ครั้งเดียวตอนจบเทอม เพราะปัญหาการ deploy มักใช้เวลาแก้ นานกว่าที่คิด หากสะสมไว้ท้ายเทอมอาจแก้ไม่ทัน

10. เกณฑ์ประเมิน/Checklist สำหรับอาจารย์ (Rubric ตัวอย่าง)

หัวข้อ	เกณฑ์	น้ำหนักคะแนนแนะนำ
ความสอดคล้องกับ User Journey	หน้าจอ/ฟีเจอร์ตรงกับขั้นตอนที่ออกแบบไว้ในเอกสารต้นทาง	20%
การทำงานของระบบ	ฟีเจอร์หลักทำงานได้จริงแบบ end-to-end (ไม่ใช่แค่หน้าตา)	25%
Backend + Database	มี API และฐานข้อมูลจริงที่เชื่อมกับ frontend ได้ถูกต้อง	20%
การจัดการ Edge Case	ระบบรองรับ Edge Case อย่างน้อย 2-3 กรณีจากเอกสารเดิม	15%
Deployment	ระบบ deploy ใช้งานได้จริงผ่าน URL สาธารณะ อัปเดตล่าสุด, docker compose up รันจากเครื่องใครก็ได้โดยไม่ error	10%
กระบวนการทำงาน (Git/Prompt)	มีประวัติ commit สม่่าเสมอ, บันทึก prompt ที่ใช้ไว้ใน README	10%

11. ข้อควรระวังเมื่อใช้ AI ช่วยเขียนโค้ด (สำหรับนิสิตมือใหม่)

- ห้าม **copy-paste** โค้ดทั้งดุ้นโดยไม่เข้าใจ — ให้ AI อธิบายทุกครั้งก่อน commit เพื่อฝึกอ่านโค้ดให้เป็น ไม่ใช่แค่ให้ AI ทำแทนทั้งหมด
- ห้าม **commit** ไฟล์ **.env** หรือคีย์ลับใดๆ ขึ้น **GitHub** — ตรวจสอบว่า **.gitignore** มี **.env** อยู่ตั้งแต่วันแรก และหากพลาด commit ไปแล้วต้องเปลี่ยนคีย์ใหม่ทันที (ไม่ใช่แค่ลบไฟล์ออก)
- **แตก prompt เป็นขั้นเล็กเสมอ** — ถ้า AI สร้างโค้ดยาวเกินไปในครั้งเดียว นิสิตจะตรวจสอบ/แก้ บั๊กไม่ทัน ควรขอทีละหน้า/ทีละฟีเจอร์

- **ตรวจสอบ Security เบื้องต้น** — โดยเฉพาะฟอร์มที่รับข้อมูลผู้ใช้ (ป้องกัน SQL Injection ผ่านการใช้ SQLAlchemy/ORM อย่างถูกต้อง ไม่ต่อ SQL string เอง, validate input ด้วย Pydantic schema ทุกจุด)
 - **จด requirements.txt ให้ครบทุกครั้ง** ที่ prompt ให้ AI ใช้ library ใหม่ — ถ้า AI แนะนำให้ `pip install` package เพิ่ม ต้องเพิ่มชื่อ+เวอร์ชันลงใน `requirements.txt` หันที ไม่เช่นนั้น Docker image ที่ build ใหม่ (รวมถึงตอน deploy จริง) จะขาด package แล้วรันไม่ได้
 - **อย่าเชื่อผลลัพธ์ของ AI 100%** — โดยเฉพาะการคำนวณ, logic ทางธุรกิจ หรือ Edge Case ที่ซับซ้อน ให้ทดสอบด้วยตัวเองเทียบกับ Acceptance Criteria ในเอกสาร User Journey เสมอ
 - **บันทึกทุก prompt สำคัญไว้** — เพื่อใช้ทบทวนกระบวนการเรียนรู้ และเป็นหลักฐานว่านิสิตเข้าใจสิ่งที่ตัวเองทำ ไม่ใช่แค่ได้ผลลัพธ์จาก AI มาโดยไม่รู้ที่มา
-